

Naive Bayes: From First Principles

Why Do We Need Naive Bayes?

Many classification problems are naturally probabilistic. Instead of asking only which class is possible, we ask which class is more likely given the evidence.

Imagine reading an email. It contains words like "win", "prize", "urgent", and "claim". A human may immediately feel that the email is probably spam. We are not using one exact rule. We are combining clues.

Naive Bayes is a classification algorithm that combines evidence using probability. It is especially famous for text classification, spam detection, sentiment analysis, and document categorization.

Probability Review

Probability measures how likely an event is. A probability is between 0 and 1. A probability of 0 means impossible. A probability of 1 means certain.

If 30 out of 100 emails are spam, then:

$$P(\textit{Spam}) = \frac{30}{100} = 0.30$$

This means the probability of spam is 30% before looking at the email content.

Conditional Probability

Conditional probability means the probability of something given that something else has happened.

For example, $P(\textit{Spam} \mid \textit{word} = \textit{win})$ means the probability that an email is spam given that the word "win" appears.

The formula is:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Here, $P(A \mid B)$ means probability of A given B. The term $P(A \cap B)$ means probability that A and B happen together.

Bayes' Theorem

Bayes' Theorem lets us reverse conditional probability. Sometimes it is easier to know how likely a word appears inside spam emails than how likely an email is spam given that word.

The formula is:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

For spam detection, we can write:

$$P(\textit{Spam} | \textit{Words}) = \frac{P(\textit{Words} | \textit{Spam})P(\textit{Spam})}{P(\textit{Words})}$$

This says: the probability of spam given the words depends on how common those words are in spam emails and how common spam is overall.

The Naive Independence Assumption

The word naive means the algorithm makes a simple assumption: features are treated as independent given the class.

In text classification, this means Naive Bayes acts as if each word contributes independently to the final class probability.

In real life, words are not fully independent. For example, "New" and "York" often appear together. But the assumption works surprisingly well in many problems.

This assumption makes the calculation much simpler.

$$P(x_1, x_2, \dots, x_n | C) = P(x_1 | C)P(x_2 | C) \cdots P(x_n | C)$$

Here, C is a class, and $x_1, x_2, \dots, x_n$ are features.

How Naive Bayes Makes Decisions

For each class, Naive Bayes calculates a score proportional to:

$$P(C | x_1, x_2, \dots, x_n) \propto P(C) \prod_{j=1}^n P(x_j | C)$$

The model chooses the class with the highest probability score.

The term $P(C)$ is the prior probability of the class. It tells us how common the class is before seeing the features.

The term $P(x_j | C)$ is the likelihood. It tells us how likely feature x_j is inside class C .

The product combines the evidence from all features.

Gaussian Naive Bayes

Gaussian Naive Bayes is used when features are continuous numbers and are assumed to follow a normal distribution inside each class.

For example, if we classify flowers using petal length and petal width, those measurements are continuous.

The Gaussian likelihood is:

$$P(x | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} e^{-\frac{(x-\mu_C)^2}{2\sigma_C^2}}$$

Here, μ_C is the mean of the feature inside class C , and σ_C^2 is the variance of the feature inside class C .

The model estimates the mean and variance for each feature in each class, then uses them to calculate probabilities.

Multinomial and Bernoulli Naive Bayes

Multinomial Naive Bayes is commonly used for count data. In text classification, it works well when features represent word counts.

For example, an email may contain the word "free" 3 times and the word "meeting" 0 times. Multinomial Naive Bayes uses these counts.

Bernoulli Naive Bayes is used when features are binary. It cares whether a word is present or absent, not how many times it appears.

For example, "free" is present or not present. "meeting" is present or not present.

Multinomial is often used for word frequency. Bernoulli is often used for word presence.

Laplace Smoothing

A major problem happens when a word never appears in a class during training. Its probability becomes zero. Since Naive Bayes multiplies probabilities, one zero can make the entire class score zero.

Laplace smoothing fixes this by adding a small count.

$$P(\text{word} | \text{class}) = \frac{\text{count}(\text{word}, \text{class}) + \alpha}{\text{total words in class} + \alpha V}$$

Here, V is the vocabulary size and α is the smoothing value. Usually $\alpha = 1$.

Smoothing prevents zero probabilities and makes the model more stable.

Worked Example

Suppose spam emails often contain "win" and normal emails often contain "meeting".

For a new email with the words "win prize", Naive Bayes calculates a spam score and a normal score.

$$Score(Spam) = P(Spam)P(win | Spam)P(prize | Spam)$$

$$Score(Normal) = P(Normal)P(win | Normal)P(prize | Normal)$$

If the spam score is larger, the model predicts Spam. If the normal score is larger, it predicts Normal.

Advantages, Disadvantages, and Evaluation

Naive Bayes is fast, simple, and works very well for many text problems. It needs relatively little data and can handle many features.

Its main weakness is the independence assumption. Features are often related in real life, so the probability estimates may not be perfectly accurate. Still, the final classification can be useful.

For classification, evaluate Naive Bayes using Confusion Matrix, Accuracy, Precision, Recall, F1-Score, and ROC-AUC when probabilities are meaningful.

Use Naive Bayes for text classification, spam detection, simple probability-based classification, and high-dimensional sparse data. Be careful when feature relationships are very important.